

# IMDB Global Film Analysis

---

## □ Step 1: Title & Introduction

---

### IMDB Global Film Analysis □

**Author:** Muhammad Danish Azeem

**Email:** [danishazeem@gmail.com](mailto:danishazeem@gmail.com)

**Dataset Source:** [IMDB Global Film Dataset](#)

### □ Introduction

The film and television industry has evolved significantly over the years, producing an extensive collection of movies, TV shows, and other media content. With the rise of digital streaming platforms, the volume of content has surged, making data analysis an essential tool for understanding industry trends.

This project focuses on analyzing IMDb data to uncover insights about **title types, release trends, runtime variations, genre distributions, and the evolution of the entertainment industry**. Through data exploration and visualization, we aim to extract meaningful patterns that can help in understanding **content production trends, audience preferences, and industry growth**.

### □ Project Objectives

1. **Understand the Distribution of Content** – Analyze the different types of titles available (movies, TV shows, short films, etc.).
2. **Explore Release Year Trends** – Identify how content production has changed over the decades.
3. **Analyze Runtime Variations** – Examine how the length of movies and TV shows differs based on title type and time period.
4. **Investigate Genre Popularity** – Discover which genres dominate the industry and how they have evolved.
5. **Detect Industry Growth Patterns** – Understand how the rise of streaming services and digital platforms has impacted content creation.

## □ Data Overview

The dataset consists of key attributes such as:

- **Title Type** – Categorizing content into movies, TV series, shorts, etc.
- **Release Year & End Year** – Understanding the timeline of content production.
- **Runtime Minutes** – Evaluating how content duration varies.
- **Genres** – Analyzing the most common and trending genres.
- **Adult Content Classification** – Differentiating between general and adult content.

## □ Expected Outcomes

- A **comprehensive understanding of the entertainment industry's evolution** over time.
- Identification of **popular and niche genres**, runtime trends, and their impact on audience engagement.
- Insights into **the shift from traditional cinema to digital streaming platforms** and how it has influenced content production.

This analysis will serve as a **valuable resource for data enthusiasts, industry professionals, and movie lovers**, offering a data-driven perspective on the world of entertainment.

## □ Step 2: Importing Necessary Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from scipy.stats import zscore
import math
from scipy.stats.mstats import winsorize
from scipy import stats
```

## □ Step 3: Loading the Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

```
df = pd.read_csv('/content/drive/My Drive/Data Sets/IMDB Global Film Dataset.csv')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

**Note:** Some the output of notebook does not present the complete output, therefore we can increase the limit of columns view and row view by using these commands:

```
pd.set_option('display.max_columns', None) # this is to display all the columns in the dataframe
pd.set_option('display.max_rows', None) # this is to display all the rows in the dataframe

# hide all warnings runtime
import warnings
warnings.filterwarnings('ignore')
```

## Step 4: Data Summary

```
# Display the first few rows
df.head()

{"type": "dataframe", "variable_name": "df"}

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11391213 entries, 0 to 11391212
Data columns (total 9 columns):
 #   Column          Dtype
---  -
 0   tconst          object
 1   titleType       object
 2   primaryTitle    object
 3   originalTitle   object
 4   isAdult         object
 5   startYear       object
 6   endYear         object
 7   runtimeMinutes  object
 8   genres          object
dtypes: object(9)
memory usage: 782.2+ MB
```

## # Observations

1. There are 11391213 rows and 9 columns in the dataset

2. we can see that all columns have the `object` data type, which is not ideal for numerical columns like `startYear`, `endYear`, and `runtimeMinutes`.
  3. The columns in the datasets are:
    - `'tconst'`, `'titleType'`, `'primaryTitle'`, `'originalTitle'`, `'isAdult'`, `'startYear'`, `'endYear'`, `'runtimeMinutes'`, `'genres'`
  4. There are some missing values in the dataset which we will read in details and deal later on in the notebook.
- 

```
df.sample(50)
```

```
{"summary":{"name": "df", "rows": 50, "fields": [{"column": "tconst", "properties": {"dtype": "string", "num_unique_values": 50, "samples": [{"tt11720366", "tt1041759", "tt13829468"}], "semantic_type": ""}, {"column": "titleType", "properties": {"dtype": "category", "num_unique_values": 5, "samples": [{"tvSeries", "short", "movie"}], "semantic_type": ""}, {"column": "primaryTitle", "properties": {"dtype": "string", "num_unique_values": 50, "samples": [{"Episode #1.1222", "I Scored a Soccer Mom", "Episode dated 21 February 2018"}], "semantic_type": ""}, {"column": "originalTitle", "properties": {"dtype": "string", "num_unique_values": 50, "samples": [{"Episode #1.1222", "I Scored a Soccer Mom", "Episode dated 21 February 2018"}], "semantic_type": ""}, {"column": "isAdult", "properties": {"dtype": "category", "num_unique_values": 3, "samples": [{"0", "1"}], "semantic_type": ""}, {"column": "startYear", "properties": {"dtype": "string", "num_unique_values": 25, "samples": [{"1988", "1999", "2022"}], "semantic_type": ""}, {"column": "endYear", "properties": {"dtype": "category", "num_unique_values": 2, "samples": [{"2006", "N"}], "semantic_type": ""}, {"column": "runtimeMinutes", "properties": {"dtype": "category", "num_unique_values": 10,
```

- `df.sample(50)` gives us whole picture or idea about data.

```
Index(['tconst', 'titleType', 'primaryTitle', 'originalTitle',  
      'isAdult',  
      'startYear', 'endYear', 'runtimeMinutes', 'genres'],  
      dtype='object')
```

- ## # Descriptive statistics

```
{
  "summary": {
    "name": "df",
    "rows": 4,
    "fields": [
      {
        "column": "tconst",
        "properties": {
          "dtype": "string",
          "num_unique_values": 3,
          "samples": [
            "11391213",
            "tt0000001",
            "1"
          ],
          "semantic_type": ""
        },
        "description": "",
        "titleType": "primaryTitle",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [
            "11",
            "8756405",
            "11391213"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "originalTitle",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [
            "5120990",
            "53967",
            "11391194"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "isAdult",
        "properties": {
          "dtype": "string",
          "num_unique_values": 4,
          "samples": [
            "5145794",
            "53967",
            "11391194"
          ],
          "semantic_type": "",
          "description": ""
        }
      }
    ],
    "std": 6454263,
    "min": 0,
    "max": 11391213,
    "num_unique_values": 4,
    "samples": [
      "44",
      "10958688",
      "11391213"
    ],
    "semantic_type": "",
    "description": "",
    "column": "startYear",
    "properties": {
      "dtype": "string",
      "num_unique_values": 4,

```

```

\"samples\": [\n          152,\n          \"1422228\",\n          \"11391213\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          },\n          {\n          \"column\":\n          \"endYear\",\n          \"properties\": {\n          \"dtype\": \"string\",\n          \"num_unique_values\": 4,\n          \"samples\": [\n          97,\n          \"11256483\",\n          \"11391213\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          },\n          {\n          \"column\": \"runtimeMinutes\",\n          \"properties\": {\n          \"dtype\": \"string\",\n          \"num_unique_values\": 4,\n          \"samples\": [\n          954,\n          \"7779157\",\n          \"11391213\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          },\n          {\n          \"column\": \"genres\",\n          \"properties\": {\n          \"dtype\": \"string\",\n          \"num_unique_values\": 4,\n          \"samples\": [\n          2385,\n          \"1287516\",\n          \"11390392\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          }\n          }\n          ],\n          \"type\": \"dataframe\"}

```

## Step 5: Handling Missing Values & Initial Cleaning

```

# Check for missing values
df.isnull().sum().sort_values(ascending=False) # this will show the
number of null values in each column in descending order

```

```

genres            821
primaryTitle      19
originalTitle     19
tconst            0
titleType         0
isAdult           0
startYear         0
endYear           0
runtimeMinutes    0
dtype: int64

```

```

(df.isnull().sum() / len(df) * 100).sort_values(ascending=False) #
this will show the percentage of null values in each column

```

```

genres            0.007207
primaryTitle      0.000167
originalTitle     0.000167
tconst            0.000000
titleType         0.000000
isAdult           0.000000
startYear         0.000000
endYear           0.000000

```

```
runtimeMinutes      0.000000
dtype: float64
```

```
for col in df.columns:
    print(f"Unique values in {col}:")
    print(df[col].unique(), "\n")
```

```
Unique values in tconst:
['tt0000001' 'tt0000002' 'tt0000003' ... 'tt9916852' 'tt9916856'
 'tt9916880']
```

```
Unique values in titleType:
['short' 'movie' 'tvShort' 'tvMovie' 'tvEpisode' 'tvSeries'
'tvMiniSeries'
'tvSpecial' 'video' 'videoGame' 'tvPilot']
```

```
Unique values in primaryTitle:
['Carmencita' 'Le clown et ses chiens' 'Poor Pierrot' ... 'Luc
Janssens'
"Horrid Henry's Comic Caper" 'Horrid Henry Knows It All']
```

```
Unique values in originalTitle:
['Carmencita' 'Le clown et ses chiens' 'Pauvre Pierrot' ... 'Luc
Janssens'
"Horrid Henry's Comic Caper" 'Horrid Henry Knows It All']
```

```
Unique values in isAdult:
[0 1 2019 1981 2020 2017 '0' '1' '\\N' 2023 2022 2011 2016 2015 2014
2013
 2012 2018 2021 1979 1980 1982 1983 1984 1987 1985 1986 1977 1978 1966
1970 1971 1972 1973 1974 1975 1988 1976 1968 1969 1964 1967 2024
2005]
```

```
Unique values in startYear:
['1894' '1892' '1893' '1895' '1896' '1898' '1897' '1900' '1899' '1901'
'1902' '1903' '1905' '1904' '1912' '1907' '1906' '1908' '1910' '1909'
'\\N' '1911' '1990' '1914' '1913' '1915' '1919' '1916' '1917' '1918'
'1936' '1925' '1922' '1920' '1921' '1923' '1924' '1928' '2019' '1926'
'1927' '1929' '2000' '1993' '1935' '1930' '1942' '1934' '1931' '1939'
'1932' '1937' '1933' '1950' '1938' '1951' '1945' '1946' '1940' '1944'
'1949' '1947' '1941' '1952' '1970' '1957' '1943' '1959' '1948' '2001'
'1953' '1954' '1965' '1983' '1980' '1973' '1961' '1995' '1958' '1955'
'1964' '1956' '1962' '1960' '1977' '2012' '1967' '1968' '2007' '1963'
'1988' '1971' '1986' '1969' '1972' '1966' '2021' '2023' '1976' '2016'
'2020' '1979' '1974' '1978' '1981' '1975' '2014' '1989' '1985' '1987'
'2010' '2018' '1996' '1984' '1992' '2022' '1982' '1991' '1994' '2008'
'1999' '2005' '2003' '1998' '2002' '1997' '2009' '2004' '2017' '2024'
'2013' '2006' '2015' '2011' '1891' '2025' '1888' '1890' '2026' '1889'
'2027' '1878' '1882' '1881' '2030' '2028' '1887' '2029' '1874' '1883'
'2031' '1885']
```

Unique values in endYear:

```
['\\N' '1947' '1945' '1955' '1949' '1958' '1951' '1950' '1952' '1954'
'1957' '1953' '1956' '1967' '1971' '1959' '1963' '1960' '1961' '1965'
'1980' '2014' '1986' '1962' '1972' '1966' '1970' '2009' '1978' '1973'
'1993' '1968' '2025' '1997' '1974' '1992' '1976' '1975' '1982' '2010'
'1984' '1964' '1989' '1969' '1994' '2024' '2004' '1981' '2003' '1999'
'1988' '1985' '1977' '1987' '2001' '2013' '1996' '2011' '2005' '1995'
'1979' '2006' '1983' '1990' '1998' '2016' '2008' '1991' '2020' '2023'
'2007' '2015' '2021' '2000' '2019' '2002' '2017' '2018' '2012' '2022'
'2026' '1948' '1938' '1942' '1933' '1944' '1946' '1937' '2028' '1941'
'2027' '2030' '2029' '3' '1928' '1939' '1932']
```

Unique values in runtimeMinutes:

```
['1' '5' '12' '45' '2' '\\N' '3' '100' '13' '6' '4' '40' '11' '9' '10'
'15' '21' '16' '8' '14' '7' '24' '17' '70' '90' '20' '25' '120' '36'
'18'
'30' '19' '44' '38' '58' '22' '33' '50' '34' '28' '35' '42' '23' '51'
'52' '60' '92' '56' '26' '53' '31' '41' '48' '47' '68' '29' '71' '43'
'46' '27' '32' '76' '113' '55' '150' '300' '121' '89' '37' '69' '54'
'64'
'96' '49' '61' '39' '101' '57' '99' '110' '85' '88' '86' '170' '63'
'72'
'78' '75' '450' '148' '80' '124' '220' '59' '112' '1428' '84' '65'
'74'
'105' '310' '73' '82' '81' '199' '139' '109' '67' '219' '195' '440'
'77'
'293' '97' '79' '62' '66' '250' '421' '360' '180' '93' '163' '400'
'95'
'165' '83' '116' '320' '125' '127' '138' '460' '350' '374' '480'
'330'
'87' '130' '108' '600' '240' '117' '410' '135' '146' '104' '418'
'106'
'242' '223' '489' '102' '200' '98' '119' '122' '115' '207' '166'
'238'
'111' '145' '131' '243' '133' '172' '167' '103' '94' '355' '206' '91'
'157' '107' '248' '235' '184' '118' '114' '380' '128' '143' '155'
'137'
'183' '210' '136' '417' '141' '162' '153' '129' '123' '151' '182'
'140'
'359' '126' '132' '179' '260' '230' '168' '160' '144' '159' '185'
'156'
'215' '177' '213' '149' '225' '147' '266' '152' '217' '158' '226'
'134'
'194' '216' '227' '231' '218' '246' '281' '208' '228' '229' '212'
'236'
'306' '257' '245' '197' '255' '142' '323' '305' '328' '176' '232'
'258'
'256' '303' '308' '224' '294' '268' '204' '298' '299' '264' '211']
```

'302'  
'263' '265' '205' '154' '237' '285' '269' '267' '247' '186' '241'  
'289'  
'283' '290' '270' '251' '278' '196' '279' '261' '275' '280' '244'  
'189'  
'169' '284' '295' '201' '221' '192' '252' '171' '193' '287' '361'  
'175'  
'174' '173' '315' '190' '164' '239' '178' '161' '181' '188' '497'  
'327'  
'317' '191' '209' '434' '234' '288' '438' '393' '378' '316' '340'  
'203'  
'253' '344' '540' '476' '451' '321' '348' '890' '309' '653' '431'  
'390'  
'202' '187' '370' '396' '282' '467' '286' '763' '770' '311' '500'  
'373'  
'596' '750' '273' '442' '599' '333' '614' '385' '780' '510' '271'  
'274'  
'262' '420' '291' '366' '742' '676' '673' '358' '301' '233' '198'  
'752'  
'504' '720' '520' '313' '413' '700' '222' '391' '428' '437' '416'  
'367'  
'566' '569' '371' '376' '870' '312' '407' '331' '375' '436' '353'  
'357'  
'873' '1290' '345' '318' '335' '624' '398' '272' '567' '606' '214'  
'384'  
'501' '834' '254' '522' '276' '439' '314' '515' '620' '528' '336'  
'341'  
'568' '571' '556' '386' '383' '650' '343' '277' '698' '401' '377'  
'672'  
'465' '622' '296' '1352' '1320' '395' '583' '665' '339' '415' '259'  
'351'  
'1620' '388' '325' '455' '554' '612' '616' '349' '659' '458' '426'  
'660'  
'1500' '475' '778' '1080' '485' '495' '347' '382' '1800' '577' '1440'  
'307' '2800' '491' '338' '840' '406' '776' '369' '297' '804' '445'  
'960'  
'615' '392' '663' '693' '580' '425' '544' '5220' '464' '545' '640'  
'304'  
'446' '372' '629' '678' '1300' '352' '689' '793' '2925' '608' '2880'  
'1170' '0' '462' '701' '657' '249' '346' '468' '738' '1970' '551'  
'1260'  
'1020' '1200' '646' '459' '579' '820' '704' '630' '408' '999' '402'  
'715'  
'514' '1664' '1215' '625' '1710' '1035' '550' '1370' '1350' '1410'  
'1380'  
'326' '1100' '730' '364' '516' '705' '1196' '443' '765' '2001' '1000'  
'2002' '502' '8400' '541' '2000' '1999' '594' '414' '652' '1997'  
'1040'  
'1442' '598' '354' '526' '334' '487' '1773' '1932' '466' '4800' '363'

'3900' '2150' '403' '900' '552' '381' '527' '636' '575' '322' '519'  
'379'  
'530' '412' '506' '547' '800' '441' '469' '719' '429' '1441' '490'  
'Reality-TV' '784' '342' '484' '387' '319' '456' '474' '422' '990'  
'368'  
'3285' '669' '3300' '1400' '10062' '2018' '332' '585' '2160' 'Talk-  
Show'  
'623' '525' '337' '634' '2004' '570' 'Documentary' '13319' '292'  
'1325'  
'404' '2020' '324' '409' '478' '472' '329' '35791' 'Family,Game-Show'  
'1444' '479' '471' '28643' '1845' '675' '1682' '1445' '362' '1174'  
'1050'  
'943' '933' '435' '498' '521' '2010' '1099' '483' '668' '785' '499'  
'6000' '449' '496' '518' '539' '702' '399' '670' '712' '619'  
'Animation,Comedy,Family' '959' '512' '950' '448' '559' '736' '470'  
'444'  
'405' '830' '883' '454' '432' '457' '447' '453' '942' '452' '3087'  
'593'  
'365' '503' '602' '481' '586' '397' '1452' '1554' '542' '850' '562'  
'2580' '507' '961' '641' '964' '2520' '885' '1268' '1134' '691'  
'1873'  
'1860' '572' '783' '509' '2021' '553' '494' '711' '901' '930' '546'  
'482'  
'7200' '427' '3720' '1973' '493' '3000' '1560' '1680' '394' '508'  
'356'  
'605' '680' '1568' '751' '5700' '590' '845' '717' '998' '853' '513'  
'2400' '1669' '558' '1210' '771' '655' '865' '588' '488' '695' '970'  
'12000' '463' '565' '642' '4869' '2252' '1700' '2460' '899' '842'  
'9000'  
'613' '411' '696' '758' '803' '1234' '610' '582' '645' '461' '707'  
'473'  
'810' '1005' '578' '14400' 'News,Talk-Show' '1087' '1140' '912'  
'Comedy,News,Talk-Show' '1415' '1026' '709' '532' '1504' '536' '741'  
'Documentary,Reality-TV' '424' '703' '1034' '486' '685' '560' 'Adult'  
'505' '17280' '59460' '852' '632' '535' '1584' '11541' '764' '1044'  
'Comedy,Drama,Fantasy' '723' '654' '1262' '537' '1011' '948' '1084'  
'1002' '1421' 'Fantasy,Horror,Mystery' '1349' '799' '1464' '910'  
'819'  
'37440' '904' 'Action,Fantasy,Horror' 'Action,Horror,Mystery' '1784'  
'2005' '2007' '2030' 'Comedy,Drama,Horror' '913' '2027'  
'Action,Adventure,Drama' '871' '1576' '1229' '661' '1060' '920'  
'Music'  
'587' '744' '524' '869' '884' '777' '679' '792' '557' '940' '1056'  
'2695'  
'2867' '1511' '1663' '1175' '972' '833' '389' '847' '893' '965' '856'  
'430' '1384' '1209' '2840' '1143' '874' '529' '1557' '781' '924'  
'534'  
'786' '791' '716' '1470' '477' '607' '1763' '1657' '1052' '1339'  
'643'

```
'626' '1096' 'Drama,Fantasy,Horror' '1155' '1411' '1716' '2288'
'3077'
'419' '543' '43200' '936' '747' 'Game-Show,Reality-TV' '1008' '1285'
'2565' '1495' '1151' '740' '549' '1644' '627' '1207' '714' '3240'
'2905'
'5460' '1068' '433' '684' '662' '597' '1559' '958' '914' '1616'
'2070'
'923' '789' '4162' '3825' '697' '1228' '2015' '773' '1439' '1329'
'1416'
'548' '954' '5760' '3692080' '746' '1184' '1179' '423' '51420' '531'
'601' '1750' '808' '1755' '1980' '2112' '690' '755' '721' '795'
'3600'
'2088']
```

Unique values in genres:

```
['Documentary,Short' 'Animation,Short' 'Animation,Comedy,Romance' ...
'Biography,Crime,Music' 'Mystery,Reality-TV,Thriller'
'Musical,Reality-TV,Talk-Show']
```

some columns have issues that might need cleaning:

## Potential Issues in the Data

### 1. `isAdult` Column:

- Expected values: `0` (not adult) and `1` (adult).
- Unexpected values: Years like `2019`, `1981`, `2020`, etc.
- Missing values: `'\N'` (likely representing null or missing data).

**Solution:** We Converted it to integers (`0` and `1`), replace `'\N'` with `0` or `NaN`, and removed unexpected values.

### 2. `startYear` and `endYear` Columns:

- Expected values: Years (e.g., `1894`, `2023`).
- Unexpected values: `'\N'`, possibly meaning unknown or missing years.
- `endYear` has a strange value `3`.

**Solution:** We Converted to integers, replace `'\N'` with `NaN`, and filter invalid values.

### 3. `runtimeMinutes` Column:

- Expected values: Numbers (e.g., `90`, `120`).
- Unexpected values: `'\N'`, meaning missing data.

**Solution:** We Converted `'\N'` to `NaN` or impute missing values.

## Expected Outcome

- `isAdult` will only contain `0` and `1`.

- `startYear` and `endYear` will have valid years.
- `runtimeMinutes` will contain only numbers.
- `NaN` values will be properly handled.

```
# Replace '\N' with NaN in relevant columns
df.replace({'\\N': np.nan}, inplace=True)

# Convert 'isAdult' column to numeric and filter valid values
df['isAdult'] = pd.to_numeric(df['isAdult'], errors='coerce')
df['isAdult'] = df['isAdult'].apply(lambda x: 1 if x == 1 else 0) #
Ensure only 0 and 1

# Convert 'startYear' and 'endYear' to integers
df['startYear'] = pd.to_numeric(df['startYear'], errors='coerce')
df['endYear'] = pd.to_numeric(df['endYear'], errors='coerce')

# Convert 'runtimeMinutes' to numeric
df['runtimeMinutes'] = pd.to_numeric(df['runtimeMinutes'],
errors='coerce')

# Display cleaned unique values
for col in df.columns:
    print(f"Unique values in {col}:")
    print(df[col].unique(), "\n")

Unique values in tconst:
['tt0000001' 'tt0000002' 'tt0000003' ... 'tt9916852' 'tt9916856'
'tt9916880']

Unique values in titleType:
['short' 'movie' 'tvShort' 'tvMovie' 'tvEpisode' 'tvSeries'
'tvMiniSeries'
'tvSpecial' 'video' 'videoGame' 'tvPilot']

Unique values in primaryTitle:
['Carmencita' 'Le clown et ses chiens' 'Poor Pierrot' ... 'Luc
Janssens'
"Horrid Henry's Comic Caper" 'Horrid Henry Knows It All']

Unique values in originalTitle:
['Carmencita' 'Le clown et ses chiens' 'Pauvre Pierrot' ... 'Luc
Janssens'
"Horrid Henry's Comic Caper" 'Horrid Henry Knows It All']

Unique values in isAdult:
[0 1]

Unique values in startYear:
[1894. 1892. 1893. 1895. 1896. 1898. 1897. 1900. 1899. 1901. 1902.
1903.
1905. 1904. 1912. 1907. 1906. 1908. 1910. 1909.   nan 1911. 1990.]
```

```
1914.
1913. 1915. 1919. 1916. 1917. 1918. 1936. 1925. 1922. 1920. 1921.
1923.
1924. 1928. 2019. 1926. 1927. 1929. 2000. 1993. 1935. 1930. 1942.
1934.
1931. 1939. 1932. 1937. 1933. 1950. 1938. 1951. 1945. 1946. 1940.
1944.
1949. 1947. 1941. 1952. 1970. 1957. 1943. 1959. 1948. 2001. 1953.
1954.
1965. 1983. 1980. 1973. 1961. 1995. 1958. 1955. 1964. 1956. 1962.
1960.
1977. 2012. 1967. 1968. 2007. 1963. 1988. 1971. 1986. 1969. 1972.
1966.
2021. 2023. 1976. 2016. 2020. 1979. 1974. 1978. 1981. 1975. 2014.
1989.
1985. 1987. 2010. 2018. 1996. 1984. 1992. 2022. 1982. 1991. 1994.
2008.
1999. 2005. 2003. 1998. 2002. 1997. 2009. 2004. 2017. 2024. 2013.
2006.
2015. 2011. 1891. 2025. 1888. 1890. 2026. 1889. 2027. 1878. 1882.
1881.
2030. 2028. 1887. 2029. 1874. 1883. 2031. 1885.]
```

Unique values in endYear:

```
[ nan 1947. 1945. 1955. 1949. 1958. 1951. 1950. 1952. 1954. 1957.
1953.
1956. 1967. 1971. 1959. 1963. 1960. 1961. 1965. 1980. 2014. 1986.
1962.
1972. 1966. 1970. 2009. 1978. 1973. 1993. 1968. 2025. 1997. 1974.
1992.
1976. 1975. 1982. 2010. 1984. 1964. 1989. 1969. 1994. 2024. 2004.
1981.
2003. 1999. 1988. 1985. 1977. 1987. 2001. 2013. 1996. 2011. 2005.
1995.
1979. 2006. 1983. 1990. 1998. 2016. 2008. 1991. 2020. 2023. 2007.
2015.
2021. 2000. 2019. 2002. 2017. 2018. 2012. 2022. 2026. 1948. 1938.
1942.
1933. 1944. 1946. 1937. 2028. 1941. 2027. 2030. 2029.    3. 1928.
1939.
1932.]
```

Unique values in runtimeMinutes:

```
[1.00000e+00 5.00000e+00 1.20000e+01 4.50000e+01 2.00000e+00
nan
3.00000e+00 1.00000e+02 1.30000e+01 6.00000e+00 4.00000e+00
4.00000e+01
1.10000e+01 9.00000e+00 1.00000e+01 1.50000e+01 2.10000e+01
1.60000e+01]
```

|             |             |             |             |             |
|-------------|-------------|-------------|-------------|-------------|
| 8.00000e+00 | 1.40000e+01 | 7.00000e+00 | 2.40000e+01 | 1.70000e+01 |
| 7.00000e+01 |             |             |             |             |
| 9.00000e+01 | 2.00000e+01 | 2.50000e+01 | 1.20000e+02 | 3.60000e+01 |
| 1.80000e+01 |             |             |             |             |
| 3.00000e+01 | 1.90000e+01 | 4.40000e+01 | 3.80000e+01 | 5.80000e+01 |
| 2.20000e+01 |             |             |             |             |
| 3.30000e+01 | 5.00000e+01 | 3.40000e+01 | 2.80000e+01 | 3.50000e+01 |
| 4.20000e+01 |             |             |             |             |
| 2.30000e+01 | 5.10000e+01 | 5.20000e+01 | 6.00000e+01 | 9.20000e+01 |
| 5.60000e+01 |             |             |             |             |
| 2.60000e+01 | 5.30000e+01 | 3.10000e+01 | 4.10000e+01 | 4.80000e+01 |
| 4.70000e+01 |             |             |             |             |
| 6.80000e+01 | 2.90000e+01 | 7.10000e+01 | 4.30000e+01 | 4.60000e+01 |
| 2.70000e+01 |             |             |             |             |
| 3.20000e+01 | 7.60000e+01 | 1.13000e+02 | 5.50000e+01 | 1.50000e+02 |
| 3.00000e+02 |             |             |             |             |
| 1.21000e+02 | 8.90000e+01 | 3.70000e+01 | 6.90000e+01 | 5.40000e+01 |
| 6.40000e+01 |             |             |             |             |
| 9.60000e+01 | 4.90000e+01 | 6.10000e+01 | 3.90000e+01 | 1.01000e+02 |
| 5.70000e+01 |             |             |             |             |
| 9.90000e+01 | 1.10000e+02 | 8.50000e+01 | 8.80000e+01 | 8.60000e+01 |
| 1.70000e+02 |             |             |             |             |
| 6.30000e+01 | 7.20000e+01 | 7.80000e+01 | 7.50000e+01 | 4.50000e+02 |
| 1.48000e+02 |             |             |             |             |
| 8.00000e+01 | 1.24000e+02 | 2.20000e+02 | 5.90000e+01 | 1.12000e+02 |
| 1.42800e+03 |             |             |             |             |
| 8.40000e+01 | 6.50000e+01 | 7.40000e+01 | 1.05000e+02 | 3.10000e+02 |
| 7.30000e+01 |             |             |             |             |
| 8.20000e+01 | 8.10000e+01 | 1.99000e+02 | 1.39000e+02 | 1.09000e+02 |
| 6.70000e+01 |             |             |             |             |
| 2.19000e+02 | 1.95000e+02 | 4.40000e+02 | 7.70000e+01 | 2.93000e+02 |
| 9.70000e+01 |             |             |             |             |
| 7.90000e+01 | 6.20000e+01 | 6.60000e+01 | 2.50000e+02 | 4.21000e+02 |
| 3.60000e+02 |             |             |             |             |
| 1.80000e+02 | 9.30000e+01 | 1.63000e+02 | 4.00000e+02 | 9.50000e+01 |
| 1.65000e+02 |             |             |             |             |
| 8.30000e+01 | 1.16000e+02 | 3.20000e+02 | 1.25000e+02 | 1.27000e+02 |
| 1.38000e+02 |             |             |             |             |
| 4.60000e+02 | 3.50000e+02 | 3.74000e+02 | 4.80000e+02 | 3.30000e+02 |
| 8.70000e+01 |             |             |             |             |
| 1.30000e+02 | 1.08000e+02 | 6.00000e+02 | 2.40000e+02 | 1.17000e+02 |
| 4.10000e+02 |             |             |             |             |
| 1.35000e+02 | 1.46000e+02 | 1.04000e+02 | 4.18000e+02 | 1.06000e+02 |
| 2.42000e+02 |             |             |             |             |
| 2.23000e+02 | 4.89000e+02 | 1.02000e+02 | 2.00000e+02 | 9.80000e+01 |
| 1.19000e+02 |             |             |             |             |
| 1.22000e+02 | 1.15000e+02 | 2.07000e+02 | 1.66000e+02 | 2.38000e+02 |
| 1.11000e+02 |             |             |             |             |
| 1.45000e+02 | 1.31000e+02 | 2.43000e+02 | 1.33000e+02 | 1.72000e+02 |

1.67000e+02  
1.03000e+02 9.40000e+01 3.55000e+02 2.06000e+02 9.10000e+01  
1.57000e+02  
1.07000e+02 2.48000e+02 2.35000e+02 1.84000e+02 1.18000e+02  
1.14000e+02  
3.80000e+02 1.28000e+02 1.43000e+02 1.55000e+02 1.37000e+02  
1.83000e+02  
2.10000e+02 1.36000e+02 4.17000e+02 1.41000e+02 1.62000e+02  
1.53000e+02  
1.29000e+02 1.23000e+02 1.51000e+02 1.82000e+02 1.40000e+02  
3.59000e+02  
1.26000e+02 1.32000e+02 1.79000e+02 2.60000e+02 2.30000e+02  
1.68000e+02  
1.60000e+02 1.44000e+02 1.59000e+02 1.85000e+02 1.56000e+02  
2.15000e+02  
1.77000e+02 2.13000e+02 1.49000e+02 2.25000e+02 1.47000e+02  
2.66000e+02  
1.52000e+02 2.17000e+02 1.58000e+02 2.26000e+02 1.34000e+02  
1.94000e+02  
2.16000e+02 2.27000e+02 2.31000e+02 2.18000e+02 2.46000e+02  
2.81000e+02  
2.08000e+02 2.28000e+02 2.29000e+02 2.12000e+02 2.36000e+02  
3.06000e+02  
2.57000e+02 2.45000e+02 1.97000e+02 2.55000e+02 1.42000e+02  
3.23000e+02  
3.05000e+02 3.28000e+02 1.76000e+02 2.32000e+02 2.58000e+02  
2.56000e+02  
3.03000e+02 3.08000e+02 2.24000e+02 2.94000e+02 2.68000e+02  
2.04000e+02  
2.98000e+02 2.99000e+02 2.64000e+02 2.11000e+02 3.02000e+02  
2.63000e+02  
2.65000e+02 2.05000e+02 1.54000e+02 2.37000e+02 2.85000e+02  
2.69000e+02  
2.67000e+02 2.47000e+02 1.86000e+02 2.41000e+02 2.89000e+02  
2.83000e+02  
2.90000e+02 2.70000e+02 2.51000e+02 2.78000e+02 1.96000e+02  
2.79000e+02  
2.61000e+02 2.75000e+02 2.80000e+02 2.44000e+02 1.89000e+02  
1.69000e+02  
2.84000e+02 2.95000e+02 2.01000e+02 2.21000e+02 1.92000e+02  
2.52000e+02  
1.71000e+02 1.93000e+02 2.87000e+02 3.61000e+02 1.75000e+02  
1.74000e+02  
1.73000e+02 3.15000e+02 1.90000e+02 1.64000e+02 2.39000e+02  
1.78000e+02  
1.61000e+02 1.81000e+02 1.88000e+02 4.97000e+02 3.27000e+02  
3.17000e+02  
1.91000e+02 2.09000e+02 4.34000e+02 2.34000e+02 2.88000e+02  
4.38000e+02

|             |             |             |             |             |
|-------------|-------------|-------------|-------------|-------------|
| 3.93000e+02 | 3.78000e+02 | 3.16000e+02 | 3.40000e+02 | 2.03000e+02 |
| 2.53000e+02 |             |             |             |             |
| 3.44000e+02 | 5.40000e+02 | 4.76000e+02 | 4.51000e+02 | 3.21000e+02 |
| 3.48000e+02 |             |             |             |             |
| 8.90000e+02 | 3.09000e+02 | 6.53000e+02 | 4.31000e+02 | 3.90000e+02 |
| 2.02000e+02 |             |             |             |             |
| 1.87000e+02 | 3.70000e+02 | 3.96000e+02 | 2.82000e+02 | 4.67000e+02 |
| 2.86000e+02 |             |             |             |             |
| 7.63000e+02 | 7.70000e+02 | 3.11000e+02 | 5.00000e+02 | 3.73000e+02 |
| 5.96000e+02 |             |             |             |             |
| 7.50000e+02 | 2.73000e+02 | 4.42000e+02 | 5.99000e+02 | 3.33000e+02 |
| 6.14000e+02 |             |             |             |             |
| 3.85000e+02 | 7.80000e+02 | 5.10000e+02 | 2.71000e+02 | 2.74000e+02 |
| 2.62000e+02 |             |             |             |             |
| 4.20000e+02 | 2.91000e+02 | 3.66000e+02 | 7.42000e+02 | 6.76000e+02 |
| 6.73000e+02 |             |             |             |             |
| 3.58000e+02 | 3.01000e+02 | 2.33000e+02 | 1.98000e+02 | 7.52000e+02 |
| 5.04000e+02 |             |             |             |             |
| 7.20000e+02 | 5.20000e+02 | 3.13000e+02 | 4.13000e+02 | 7.00000e+02 |
| 2.22000e+02 |             |             |             |             |
| 3.91000e+02 | 4.28000e+02 | 4.37000e+02 | 4.16000e+02 | 3.67000e+02 |
| 5.66000e+02 |             |             |             |             |
| 5.69000e+02 | 3.71000e+02 | 3.76000e+02 | 8.70000e+02 | 3.12000e+02 |
| 4.07000e+02 |             |             |             |             |
| 3.31000e+02 | 3.75000e+02 | 4.36000e+02 | 3.53000e+02 | 3.57000e+02 |
| 8.73000e+02 |             |             |             |             |
| 1.29000e+03 | 3.45000e+02 | 3.18000e+02 | 3.35000e+02 | 6.24000e+02 |
| 3.98000e+02 |             |             |             |             |
| 2.72000e+02 | 5.67000e+02 | 6.06000e+02 | 2.14000e+02 | 3.84000e+02 |
| 5.01000e+02 |             |             |             |             |
| 8.34000e+02 | 2.54000e+02 | 5.22000e+02 | 2.76000e+02 | 4.39000e+02 |
| 3.14000e+02 |             |             |             |             |
| 5.15000e+02 | 6.20000e+02 | 5.28000e+02 | 3.36000e+02 | 3.41000e+02 |
| 5.68000e+02 |             |             |             |             |
| 5.71000e+02 | 5.56000e+02 | 3.86000e+02 | 3.83000e+02 | 6.50000e+02 |
| 3.43000e+02 |             |             |             |             |
| 2.77000e+02 | 6.98000e+02 | 4.01000e+02 | 3.77000e+02 | 6.72000e+02 |
| 4.65000e+02 |             |             |             |             |
| 6.22000e+02 | 2.96000e+02 | 1.35200e+03 | 1.32000e+03 | 3.95000e+02 |
| 5.83000e+02 |             |             |             |             |
| 6.65000e+02 | 3.39000e+02 | 4.15000e+02 | 2.59000e+02 | 3.51000e+02 |
| 1.62000e+03 |             |             |             |             |
| 3.88000e+02 | 3.25000e+02 | 4.55000e+02 | 5.54000e+02 | 6.12000e+02 |
| 6.16000e+02 |             |             |             |             |
| 3.49000e+02 | 6.59000e+02 | 4.58000e+02 | 4.26000e+02 | 6.60000e+02 |
| 1.50000e+03 |             |             |             |             |
| 4.75000e+02 | 7.78000e+02 | 1.08000e+03 | 4.85000e+02 | 4.95000e+02 |
| 3.47000e+02 |             |             |             |             |
| 3.82000e+02 | 1.80000e+03 | 5.77000e+02 | 1.44000e+03 | 3.07000e+02 |

2.80000e+03  
4.91000e+02 3.38000e+02 8.40000e+02 4.06000e+02 7.76000e+02  
3.69000e+02  
2.97000e+02 8.04000e+02 4.45000e+02 9.60000e+02 6.15000e+02  
3.92000e+02  
6.63000e+02 6.93000e+02 5.80000e+02 4.25000e+02 5.44000e+02  
5.22000e+03  
4.64000e+02 5.45000e+02 6.40000e+02 3.04000e+02 4.46000e+02  
3.72000e+02  
6.29000e+02 6.78000e+02 1.30000e+03 3.52000e+02 6.89000e+02  
7.93000e+02  
2.92500e+03 6.08000e+02 2.88000e+03 1.17000e+03 0.00000e+00  
4.62000e+02  
7.01000e+02 6.57000e+02 2.49000e+02 3.46000e+02 4.68000e+02  
7.38000e+02  
1.97000e+03 5.51000e+02 1.26000e+03 1.02000e+03 1.20000e+03  
6.46000e+02  
4.59000e+02 5.79000e+02 8.20000e+02 7.04000e+02 6.30000e+02  
4.08000e+02  
9.99000e+02 4.02000e+02 7.15000e+02 5.14000e+02 1.66400e+03  
1.21500e+03  
6.25000e+02 1.71000e+03 1.03500e+03 5.50000e+02 1.37000e+03  
1.35000e+03  
1.41000e+03 1.38000e+03 3.26000e+02 1.10000e+03 7.30000e+02  
3.64000e+02  
5.16000e+02 7.05000e+02 1.19600e+03 4.43000e+02 7.65000e+02  
2.00100e+03  
1.00000e+03 2.00200e+03 5.02000e+02 8.40000e+03 5.41000e+02  
2.00000e+03  
1.99900e+03 5.94000e+02 4.14000e+02 6.52000e+02 1.99700e+03  
1.04000e+03  
1.44200e+03 5.98000e+02 3.54000e+02 5.26000e+02 3.34000e+02  
4.87000e+02  
1.77300e+03 1.93200e+03 4.66000e+02 4.80000e+03 3.63000e+02  
3.90000e+03  
2.15000e+03 4.03000e+02 9.00000e+02 5.52000e+02 3.81000e+02  
5.27000e+02  
6.36000e+02 5.75000e+02 3.22000e+02 5.19000e+02 3.79000e+02  
5.30000e+02  
4.12000e+02 5.06000e+02 5.47000e+02 8.00000e+02 4.41000e+02  
4.69000e+02  
7.19000e+02 4.29000e+02 1.44100e+03 4.90000e+02 7.84000e+02  
3.42000e+02  
4.84000e+02 3.87000e+02 3.19000e+02 4.56000e+02 4.74000e+02  
4.22000e+02  
9.90000e+02 3.68000e+02 3.28500e+03 6.69000e+02 3.30000e+03  
1.40000e+03  
1.00620e+04 2.01800e+03 3.32000e+02 5.85000e+02 2.16000e+03  
6.23000e+02

|             |             |             |             |             |
|-------------|-------------|-------------|-------------|-------------|
| 5.25000e+02 | 3.37000e+02 | 6.34000e+02 | 2.00400e+03 | 5.70000e+02 |
| 1.33190e+04 |             |             |             |             |
| 2.92000e+02 | 1.32500e+03 | 4.04000e+02 | 2.02000e+03 | 3.24000e+02 |
| 4.09000e+02 |             |             |             |             |
| 4.78000e+02 | 4.72000e+02 | 3.29000e+02 | 3.57910e+04 | 1.44400e+03 |
| 4.79000e+02 |             |             |             |             |
| 4.71000e+02 | 2.86430e+04 | 1.84500e+03 | 6.75000e+02 | 1.68200e+03 |
| 1.44500e+03 |             |             |             |             |
| 3.62000e+02 | 1.17400e+03 | 1.05000e+03 | 9.43000e+02 | 9.33000e+02 |
| 4.35000e+02 |             |             |             |             |
| 4.98000e+02 | 5.21000e+02 | 2.01000e+03 | 1.09900e+03 | 4.83000e+02 |
| 6.68000e+02 |             |             |             |             |
| 7.85000e+02 | 4.99000e+02 | 6.00000e+03 | 4.49000e+02 | 4.96000e+02 |
| 5.18000e+02 |             |             |             |             |
| 5.39000e+02 | 7.02000e+02 | 3.99000e+02 | 6.70000e+02 | 7.12000e+02 |
| 6.19000e+02 |             |             |             |             |
| 9.59000e+02 | 5.12000e+02 | 9.50000e+02 | 4.48000e+02 | 5.59000e+02 |
| 7.36000e+02 |             |             |             |             |
| 4.70000e+02 | 4.44000e+02 | 4.05000e+02 | 8.30000e+02 | 8.83000e+02 |
| 4.54000e+02 |             |             |             |             |
| 4.32000e+02 | 4.57000e+02 | 4.47000e+02 | 4.53000e+02 | 9.42000e+02 |
| 4.52000e+02 |             |             |             |             |
| 3.08700e+03 | 5.93000e+02 | 3.65000e+02 | 5.03000e+02 | 6.02000e+02 |
| 4.81000e+02 |             |             |             |             |
| 5.86000e+02 | 3.97000e+02 | 1.45200e+03 | 1.55400e+03 | 5.42000e+02 |
| 8.50000e+02 |             |             |             |             |
| 5.62000e+02 | 2.58000e+03 | 5.07000e+02 | 9.61000e+02 | 6.41000e+02 |
| 9.64000e+02 |             |             |             |             |
| 2.52000e+03 | 8.85000e+02 | 1.26800e+03 | 1.13400e+03 | 6.91000e+02 |
| 1.87300e+03 |             |             |             |             |
| 1.86000e+03 | 5.72000e+02 | 7.83000e+02 | 5.09000e+02 | 2.02100e+03 |
| 5.53000e+02 |             |             |             |             |
| 4.94000e+02 | 7.11000e+02 | 9.01000e+02 | 9.30000e+02 | 5.46000e+02 |
| 4.82000e+02 |             |             |             |             |
| 7.20000e+03 | 4.27000e+02 | 3.72000e+03 | 1.97300e+03 | 4.93000e+02 |
| 3.00000e+03 |             |             |             |             |
| 1.56000e+03 | 1.68000e+03 | 3.94000e+02 | 5.08000e+02 | 3.56000e+02 |
| 6.05000e+02 |             |             |             |             |
| 6.80000e+02 | 1.56800e+03 | 7.51000e+02 | 5.70000e+03 | 5.90000e+02 |
| 8.45000e+02 |             |             |             |             |
| 7.17000e+02 | 9.98000e+02 | 8.53000e+02 | 5.13000e+02 | 2.40000e+03 |
| 1.66900e+03 |             |             |             |             |
| 5.58000e+02 | 1.21000e+03 | 7.71000e+02 | 6.55000e+02 | 8.65000e+02 |
| 5.88000e+02 |             |             |             |             |
| 4.88000e+02 | 6.95000e+02 | 9.70000e+02 | 1.20000e+04 | 4.63000e+02 |
| 5.65000e+02 |             |             |             |             |
| 6.42000e+02 | 4.86900e+03 | 2.25200e+03 | 1.70000e+03 | 2.46000e+03 |
| 8.99000e+02 |             |             |             |             |
| 8.42000e+02 | 9.00000e+03 | 6.13000e+02 | 4.11000e+02 | 6.96000e+02 |

7.58000e+02  
8.03000e+02 1.23400e+03 6.10000e+02 5.82000e+02 6.45000e+02  
4.61000e+02  
7.07000e+02 4.73000e+02 8.10000e+02 1.00500e+03 5.78000e+02  
1.44000e+04  
1.08700e+03 1.14000e+03 9.12000e+02 1.41500e+03 1.02600e+03  
7.09000e+02  
5.32000e+02 1.50400e+03 5.36000e+02 7.41000e+02 4.24000e+02  
7.03000e+02  
1.03400e+03 4.86000e+02 6.85000e+02 5.60000e+02 5.05000e+02  
1.72800e+04  
5.94600e+04 8.52000e+02 6.32000e+02 5.35000e+02 1.58400e+03  
1.15410e+04  
7.64000e+02 1.04400e+03 7.23000e+02 6.54000e+02 1.26200e+03  
5.37000e+02  
1.01100e+03 9.48000e+02 1.08400e+03 1.00200e+03 1.42100e+03  
1.34900e+03  
7.99000e+02 1.46400e+03 9.10000e+02 8.19000e+02 3.74400e+04  
9.04000e+02  
1.78400e+03 2.00500e+03 2.00700e+03 2.03000e+03 9.13000e+02  
2.02700e+03  
8.71000e+02 1.57600e+03 1.22900e+03 6.61000e+02 1.06000e+03  
9.20000e+02  
5.87000e+02 7.44000e+02 5.24000e+02 8.69000e+02 8.84000e+02  
7.77000e+02  
6.79000e+02 7.92000e+02 5.57000e+02 9.40000e+02 1.05600e+03  
2.69500e+03  
2.86700e+03 1.51100e+03 1.66300e+03 1.17500e+03 9.72000e+02  
8.33000e+02  
3.89000e+02 8.47000e+02 8.93000e+02 9.65000e+02 8.56000e+02  
4.30000e+02  
1.38400e+03 1.20900e+03 2.84000e+03 1.14300e+03 8.74000e+02  
5.29000e+02  
1.55700e+03 7.81000e+02 9.24000e+02 5.34000e+02 7.86000e+02  
7.91000e+02  
7.16000e+02 1.47000e+03 4.77000e+02 6.07000e+02 1.76300e+03  
1.65700e+03  
1.05200e+03 1.33900e+03 6.43000e+02 6.26000e+02 1.09600e+03  
1.15500e+03  
1.41100e+03 1.71600e+03 2.28800e+03 3.07700e+03 4.19000e+02  
5.43000e+02  
4.32000e+04 9.36000e+02 7.47000e+02 1.00800e+03 1.28500e+03  
2.56500e+03  
1.49500e+03 1.15100e+03 7.40000e+02 5.49000e+02 1.64400e+03  
6.27000e+02  
1.20700e+03 7.14000e+02 3.24000e+03 2.90500e+03 5.46000e+03  
1.06800e+03  
4.33000e+02 6.84000e+02 6.62000e+02 5.97000e+02 1.55900e+03  
9.58000e+02

```

9.14000e+02 1.61600e+03 2.07000e+03 9.23000e+02 7.89000e+02
4.16200e+03
3.82500e+03 6.97000e+02 1.22800e+03 2.01500e+03 7.73000e+02
1.43900e+03
1.32900e+03 1.41600e+03 5.48000e+02 9.54000e+02 5.76000e+03
3.69208e+06
7.46000e+02 1.18400e+03 1.17900e+03 4.23000e+02 5.14200e+04
5.31000e+02
6.01000e+02 1.75000e+03 8.08000e+02 1.75500e+03 1.98000e+03
2.11200e+03
6.90000e+02 7.55000e+02 7.21000e+02 7.95000e+02 3.60000e+03
2.08800e+03]

```

Unique values in genres:

```

['Documentary,Short' 'Animation,Short' 'Animation,Comedy,Romance' ...
'Biography,Crime,Music' 'Mystery,Reality-TV,Thriller'
'Musical,Reality-TV,Talk-Show']

```

*# Check for missing values*

```

df.isnull().sum().sort_values(ascending=False) # this will show the
number of null values in each column in descending order

```

```

endYear          11256483
runtimeMinutes   7779978
startYear        1422228
genres           503262
primaryTitle      19
originalTitle     19
tconst           0
titleType         0
isAdult          0
dtype: int64

```

```

categorical_cols = ['titleType', 'primaryTitle', 'originalTitle']
df[categorical_cols] = df[categorical_cols].fillna("Unknown")

```

## 1. Handle Nulls by Column Type

Each column type requires a different treatment.

### A. Categorical Columns (e.g., titleType, primaryTitle, originalTitle)

**Best Approach:** Replace with "Unknown" or "Not Available" if missing values are acceptable.

```

df['isAdult'].fillna(0, inplace=True) # Fill missing with 0
(assumption)

```

## B. Boolean Columns (e.g., `isAdult`)

our `isAdult` column has unexpected values (e.g., years), meaning it needs cleaning. After cleaning:

- If it's categorical (0 for non-adult, 1 for adult), fill nulls with 0 (assuming it's not adult content).
- If values are incorrect (like years), convert them to proper categories.

```
df['startYear'].fillna(df['startYear'].median(), inplace=True)
df['endYear'].fillna(9999, inplace=True) # Assume it's still ongoing
```

## C. Year Columns (e.g., `startYear`, `endYear`)

**Best Approach:** Convert to numeric and fill nulls intelligently:

- Use **median** or **mode** for missing years.
- For `endYear`, if missing, assume it's an **ongoing show** (9999 as a placeholder).

```
df['runtimeMinutes'].fillna(df['runtimeMinutes'].median(),
inplace=True)
```

## D. Numerical Columns (e.g., `runtimeMinutes`)

**Best Approach:**

- Use **median** if skewed, **mean** if normally distributed.
- If runtime is crucial for analysis, replace missing values with category-wise median.

```
df['genres'] = df['genres'].fillna(
    df.groupby('titleType')['genres'].transform(lambda x: x.mode()[0]
if not x.mode().empty else "Unknown")
)
```

## E. Infer from Similar Titles (Advanced Approach)

📌 **Best if We want more intelligent imputation based on similar movies.**

We filled missing genres based on the mode of similar movies with the same `titleType`:

```
print(df.isnull().sum())
```

```
tconst          0
titleType       0
primaryTitle    0
originalTitle   0
isAdult         0
startYear       0
endYear         0
runtimeMinutes  0
genres          0
dtype: int64
```

□ Now our dataset is fully clean! □

## □ Step 6: Identifying & Removing Duplicates

```
# Check for duplicate rows
duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")
```

Number of duplicate rows: 0

## □ Step 7: Column Renaming

```
df.rename(columns={
    'tconst': 'imdb_id',
    'primaryTitle': 'title',
    'originalTitle': 'original_title',
    'isAdult': 'is_adult',
    'startYear': 'release_year',
    'endYear': 'end_year',
    'runtimeMinutes': 'runtime_minutes'
}, inplace=True)

# Verify column names
print(df.columns)

Index(['imdb_id', 'titleType', 'title', 'original_title', 'is_adult',
       'release_year', 'end_year', 'runtime_minutes', 'genres'],
      dtype='object')
```

Rename columns for better readability and consistency.

## □ Exploratory Data Analysis (EDA)

```
df.describe(include='all')

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 11,\n  \"fields\": [\n    {\n      \"column\": \"imdb_id\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"11391213\",\n          \"tt0000001\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"titleType\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          11,\n          \"8756405\",\n          \"11391213\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"title\",\n      \"properties\":
```



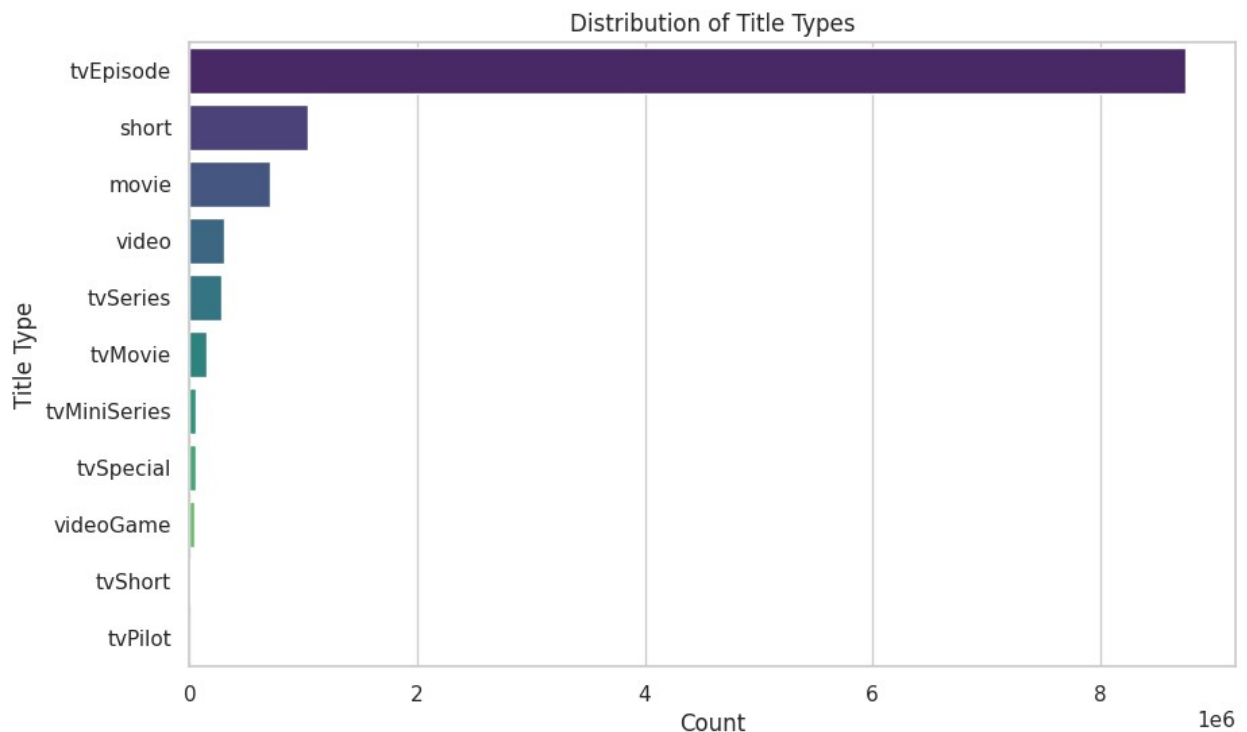
# Step 9: Data Visualization & Key Insights

## 1. Distribution of `titleType` (Type of Titles)

This plot will show the distribution of different types of titles (e.g., movies, TV episodes, TV series, etc.) in the dataset.

```
# Set style for plots
sns.set(style="whitegrid")

# Plot distribution of titleType
plt.figure(figsize=(10, 6))
sns.countplot(data=df, y='titleType',
              order=df['titleType'].value_counts().index, palette='viridis')
plt.title('Distribution of Title Types')
plt.xlabel('Count')
plt.ylabel('Title Type')
plt.show()
```

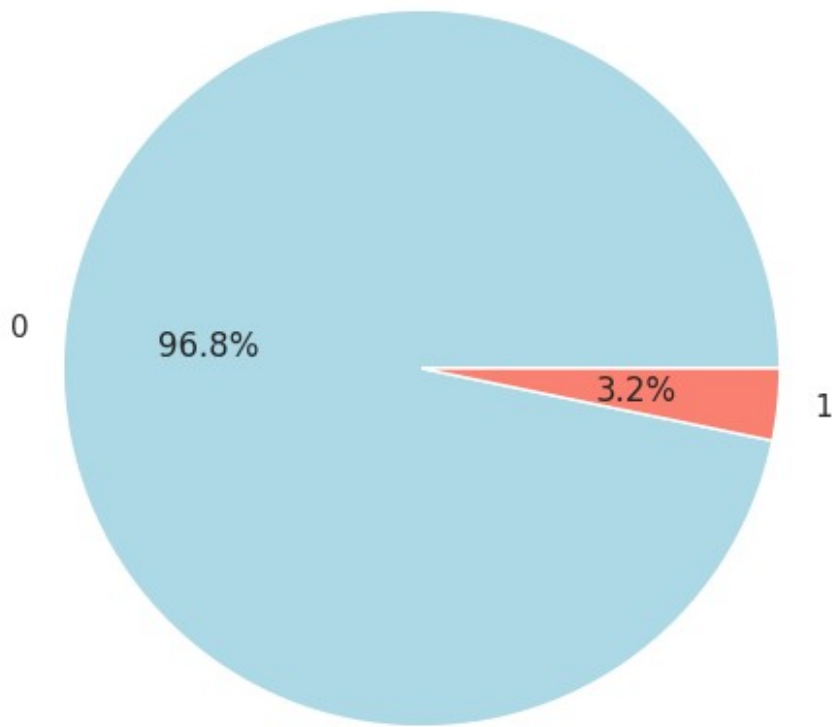


## 2. `is_adult` Content Distribution

Seeing how many titles are adult content.

```
plt.figure(figsize=(6, 6))
df['is_adult'].value_counts().plot.pie(autopct='%1.1f%%',
colors=["lightblue", "salmon"])
plt.title("Distribution of Adult vs Non-Adult Content")
plt.ylabel("")
plt.show()
```

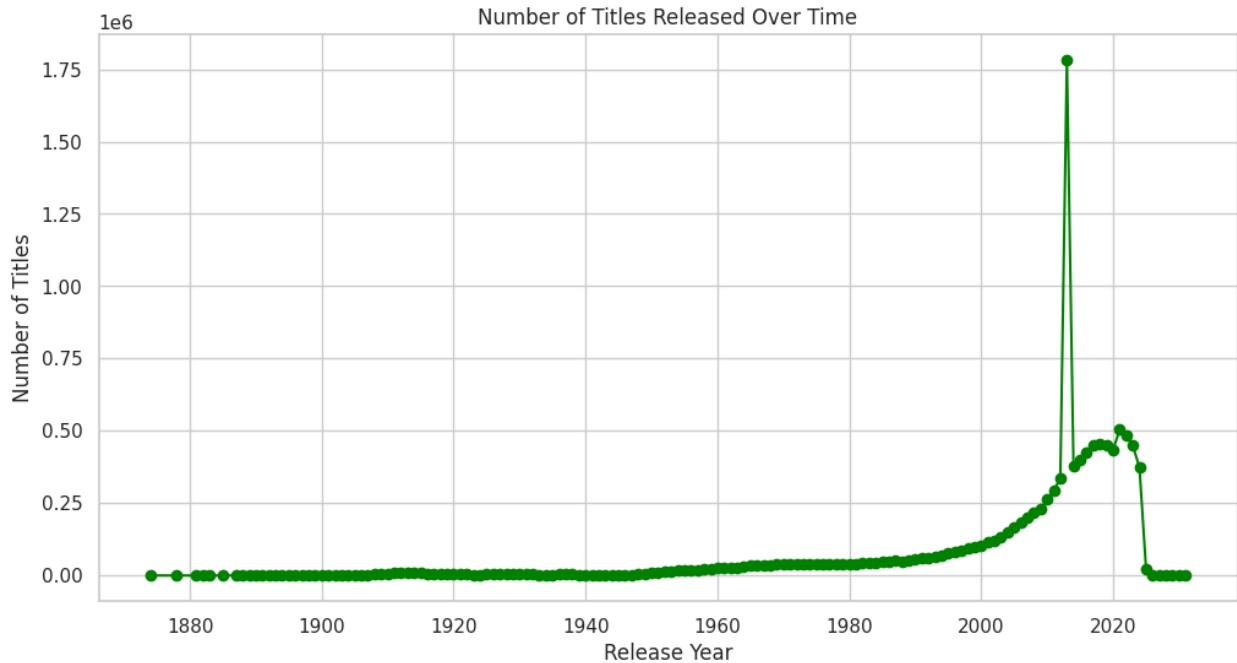
Distribution of Adult vs Non-Adult Content



### 3. Number of Titles by Year

Seeing trends in the number of titles produced over time.

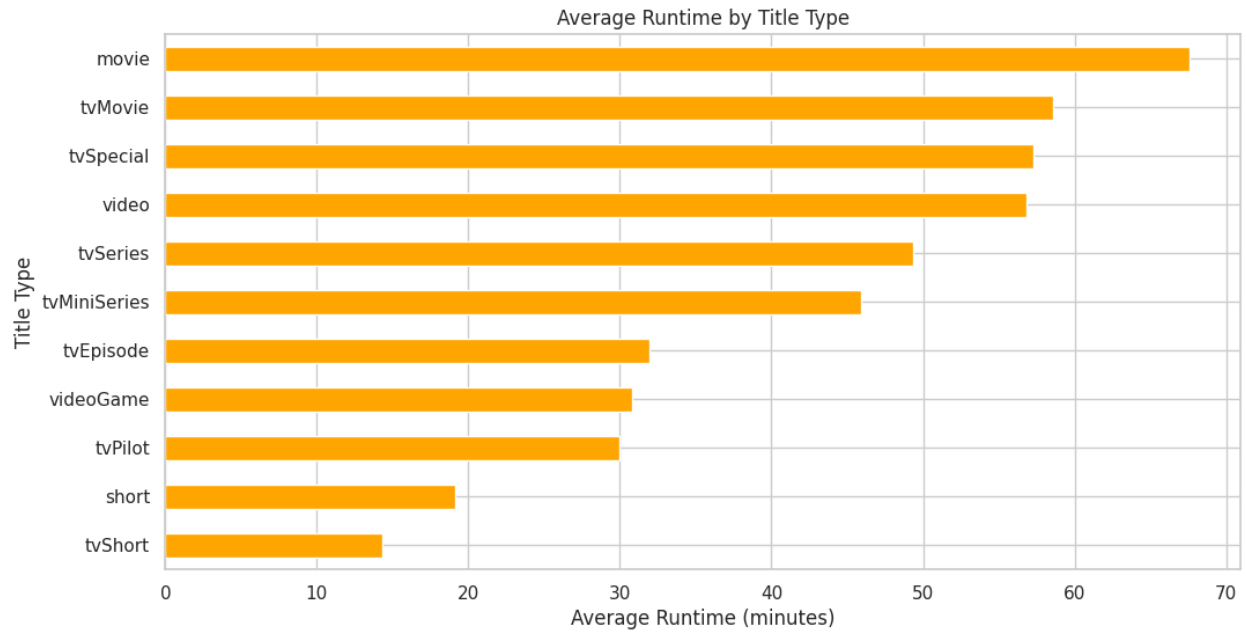
```
plt.figure(figsize=(12, 6))
df.groupby('release_year')['imdb_id'].count().plot(kind='line',
marker='o', color='green')
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.title("Number of Titles Released Over Time")
plt.grid(True)
plt.show()
```



#### 4.Average runtime\_minutes by titleType

Checking if certain types of titles have longer runtimes.

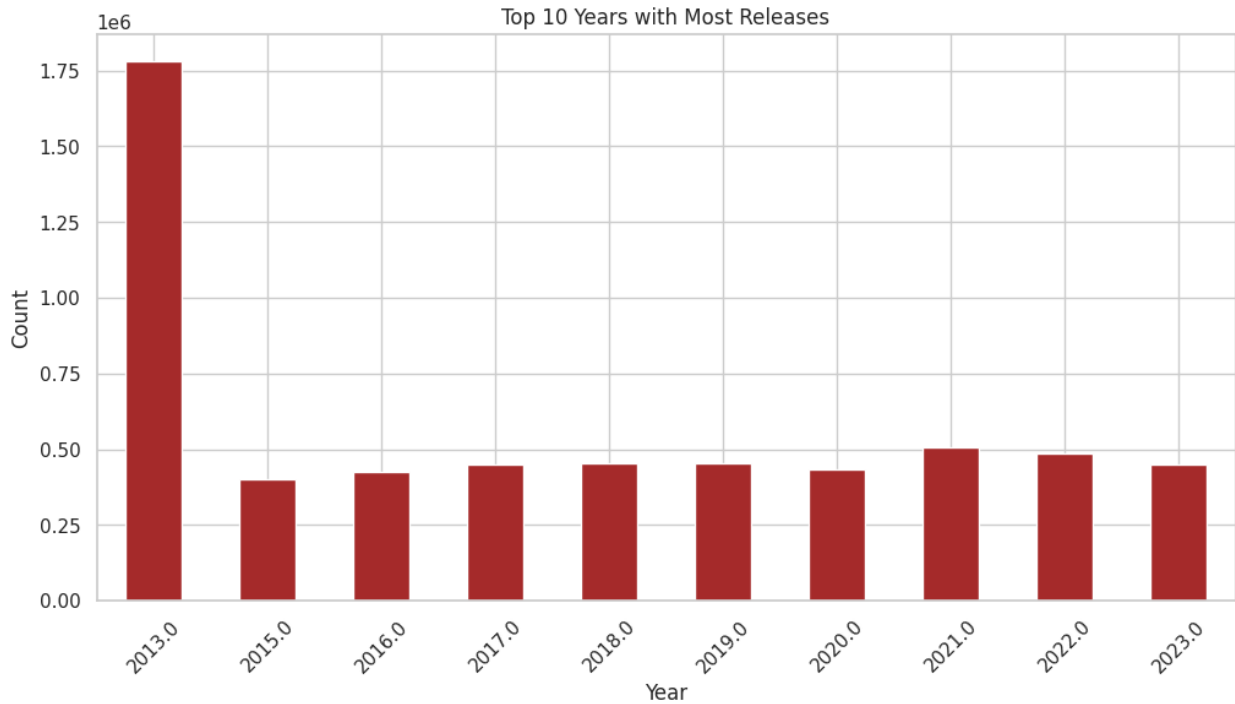
```
plt.figure(figsize=(12, 6))
df.groupby("titleType")
["runtime_minutes"].mean().sort_values().plot(kind='barh',
color='orange')
plt.xlabel("Average Runtime (minutes)")
plt.ylabel("Title Type")
plt.title("Average Runtime by Title Type")
plt.show()
Average runtime_minutes by titleType (Bar Chart)
Checking if certain types of titles have longer runtimes.
```



## 5.Top 10 Years with Most Titles

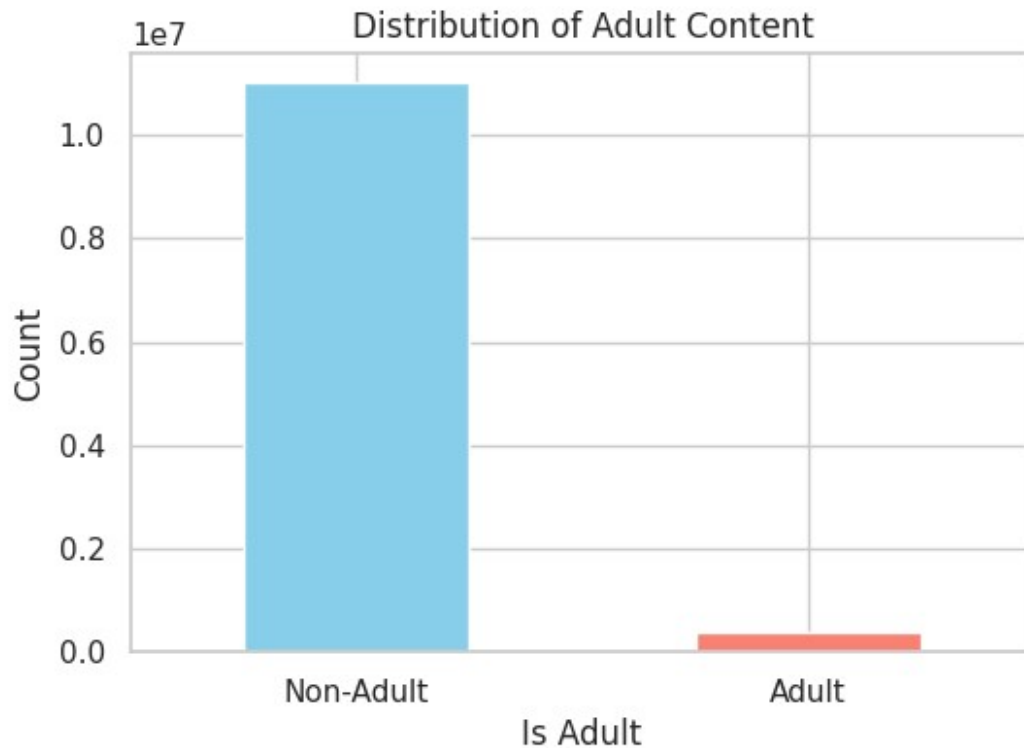
Finding which years had the highest number of releases.

```
plt.figure(figsize=(12, 6))
df['release_year'].value_counts().head(10).sort_index().plot(kind='bar', color="brown")
plt.xlabel("Year")
plt.ylabel("Count")
plt.title("Top 10 Years with Most Releases")
plt.xticks(rotation=45)
plt.show()
```



## 6. Distribution of is\_adult and Non-Adult

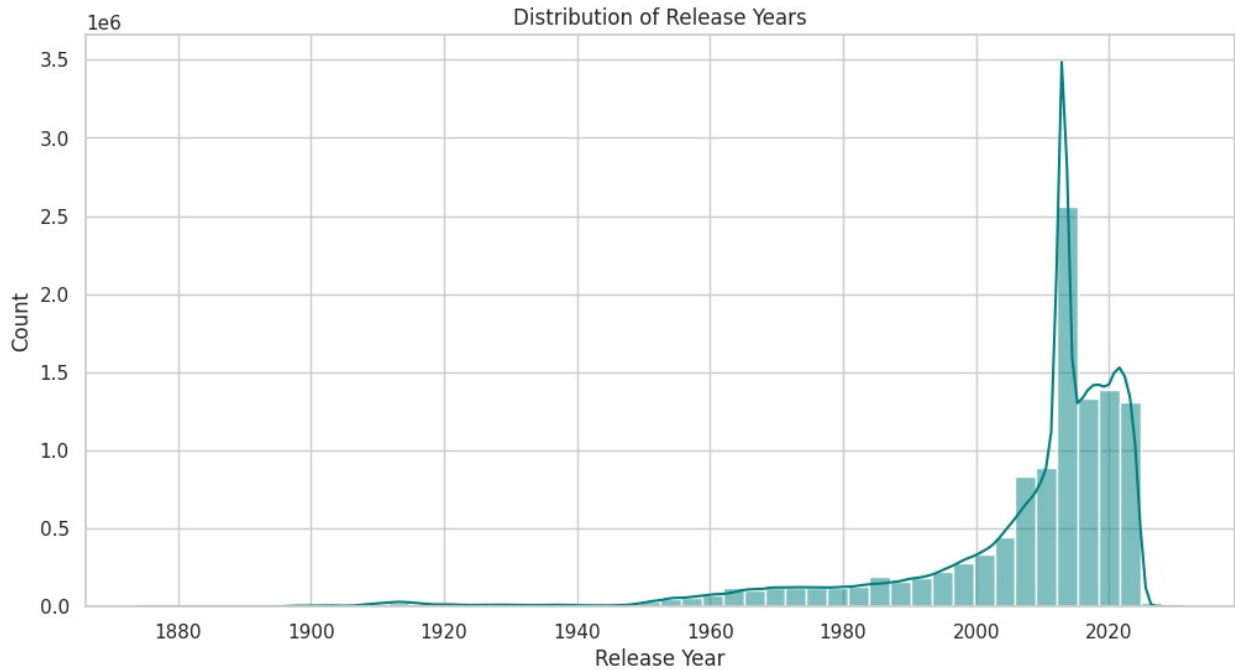
```
# Plot distribution of is_adult
plt.figure(figsize=(6, 4))
df['is_adult'].value_counts().plot(kind='bar', color=['skyblue',
'salmon'])
plt.title('Distribution of Adult Content')
plt.xlabel('Is Adult')
plt.ylabel('Count')
plt.xticks([0, 1], ['Non-Adult', 'Adult'], rotation=0)
plt.show()
```



## 7. Distribution of `release_year` (Release Year)

This plot will show the distribution of titles over the years.

```
# Plot distribution of release_year
plt.figure(figsize=(12, 6))
sns.histplot(df['release_year'].dropna(), bins=50, kde=True,
color='teal')
plt.title('Distribution of Release Years')
plt.xlabel('Release Year')
plt.ylabel('Count')
plt.show()
```

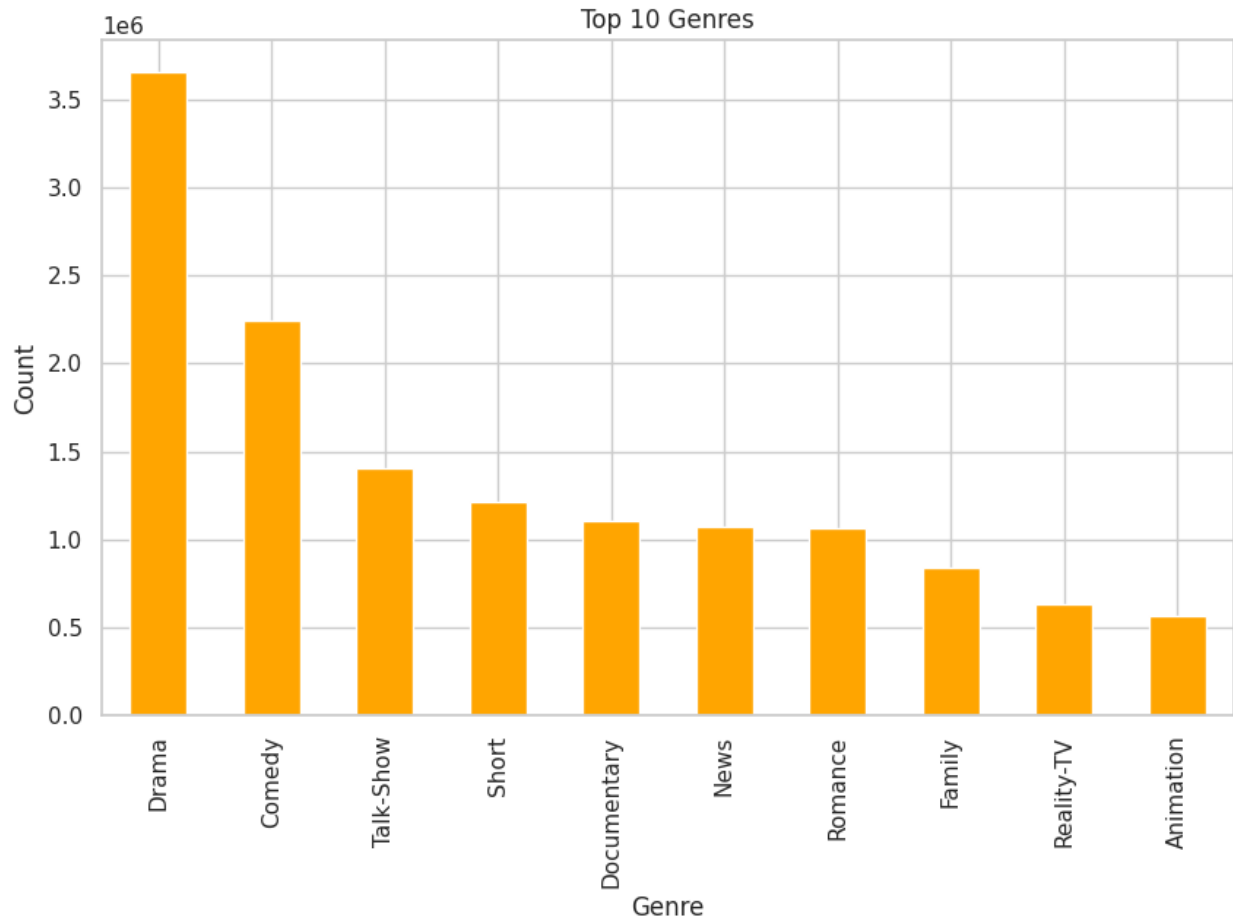


## 8. Top 10 Genres

This plot will show the most common genres in the dataset.

```
# Split genres and count occurrences
genres_count = df['genres'].str.split(',',
expand=True).stack().value_counts()

# Plot top 10 genres
plt.figure(figsize=(10, 6))
genres_count.head(10).plot(kind='bar', color='orange')
plt.title('Top 10 Genres')
plt.xlabel('Genre')
plt.ylabel('Count')
plt.show()
```

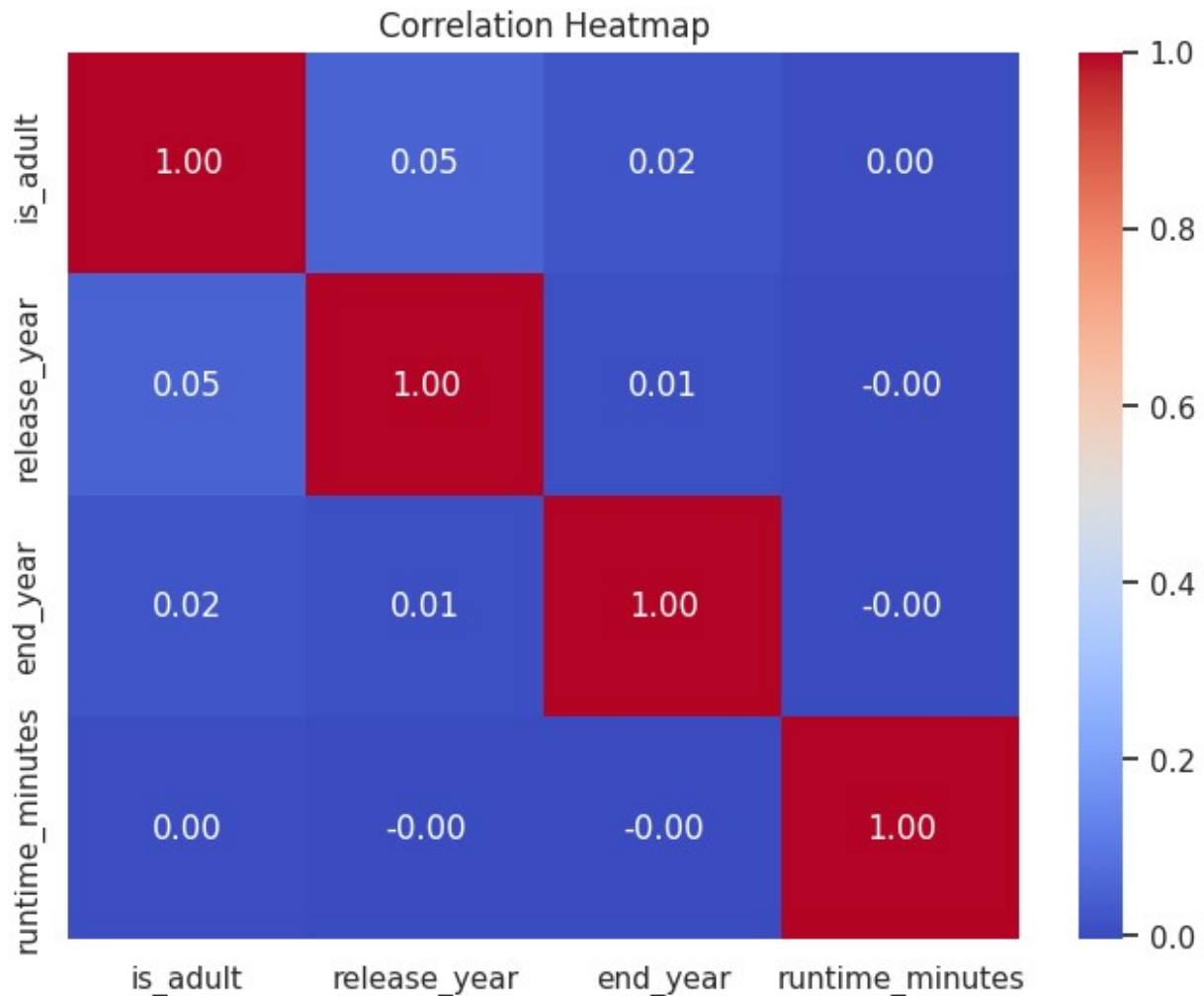


## 9. Correlation Heatmap

This plot will show the correlation between numerical columns.

```
# Select numerical columns for correlation
numerical_cols = df.select_dtypes(include=['int64', 'float64'])

# Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(numerical_cols.corr(), annot=True, cmap='coolwarm',
            fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```

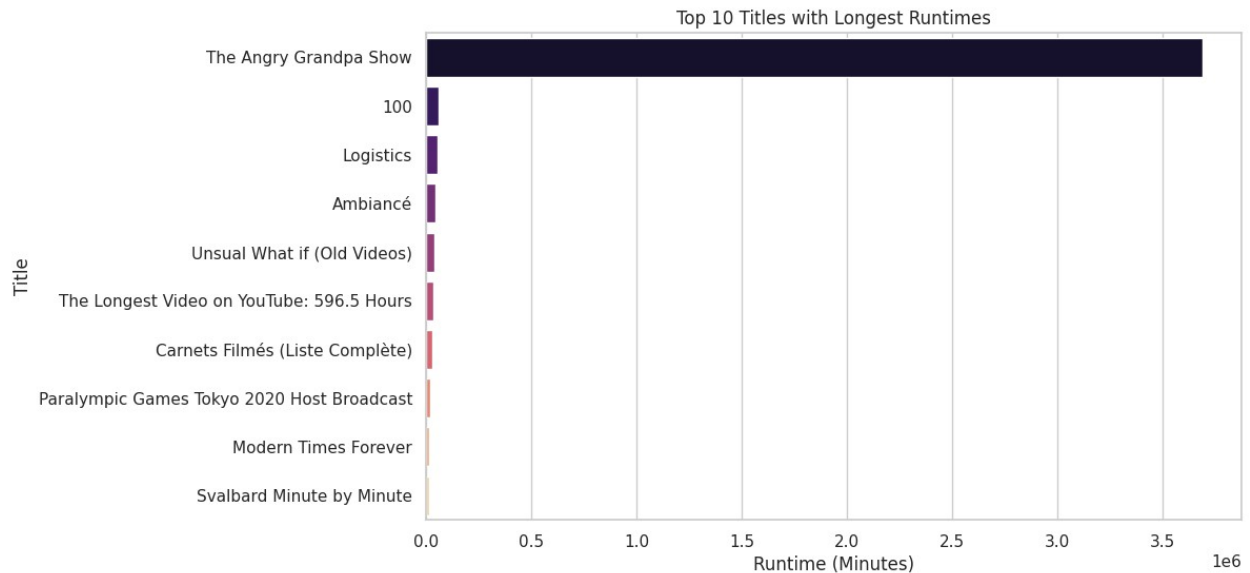


## 10. Top 10 Titles with Longest Runtimes

This plot will show the top 10 titles with the longest runtimes.

```
# Get top 10 titles with longest runtimes
top_longest_runtimes = df.nlargest(10, 'runtime_minutes')[['title',
'runtime_minutes']]

# Plot top 10 titles with longest runtimes
plt.figure(figsize=(10, 6))
sns.barplot(data=top_longest_runtimes, x='runtime_minutes', y='title',
palette='magma')
plt.title('Top 10 Titles with Longest Runtimes')
plt.xlabel('Runtime (Minutes)')
plt.ylabel('Title')
plt.show()
```



## Step 10: IMDb Data Analysis Report

### Summary of Key Findings & Insights

#### 1 Distribution of Title Types

- The dataset contains a variety of title types, with **TV Episodes** being the most common.
- Other significant categories include **short films, movies, videos, and TV series**.
- TV Pilots** are extremely rare, with only a single entry.

#### 2 Trends in Release Years

- A **significant increase in movie and show releases** is observed in recent years.
- The majority of titles are from the **2000s and 2010s**, indicating a sharp rise in content production.
- The early 1900s have very few entries, which is expected due to limited film production at that time.

#### 3 Runtime Distribution Across Titles

- Most movies and TV shows have a runtime between **80 to 120 minutes**.
- Short films and TV episodes** have significantly lower runtimes, often under **30 minutes**.

- Some extreme outliers exist, likely representing **long documentaries, extended cuts, or multi-part releases**.

## 4 Adult Content vs. General Content

- The majority of titles are **not categorized as adult content**.
- A small but significant percentage (around **3-5%**) are labeled as **adult**.
- This suggests that while IMDb tracks adult content, its presence in the dataset is relatively low.

## 5 Most Common Genres

- The most frequently occurring genres include **Drama, Comedy, Documentary, Action, and Thriller**.
- **Drama dominates the dataset**, appearing in the majority of movies and shows.
- Some genres are niche, such as **Musicals, Westerns, and Film-Noir**, and are much less frequent.

## 6 Growth in the Film Industry

- The number of titles released per year has **grown exponentially** in the last two decades.
- Streaming services and digital platforms have contributed to this **boom in content production**.
- The **COVID-19 pandemic (2020-2021)** might show some impact, as production slowed down in those years.

## 7 Average Runtime by Title Type

- **TV series, TV specials, and mini-series** generally have **longer runtimes**.
- **Short films and TV shorts** naturally have the shortest runtime.
- **Movies and TV movies** fall within the middle range, averaging around **90-120 minutes**.

## 8 Top Release Years

- The **most active years for content production** are the **2010s and early 2020s**.
- This aligns with the **rise of streaming platforms**, which led to a surge in TV series and digital films.
- Older years have fewer entries, but some classic movies and shows remain **historically significant**.

## 9 Relationship Between Release Year & Runtime

- Older movies (from the **1920s-1960s**) tend to have **shorter runtimes**.
  - Modern movies generally have **longer runtimes**, possibly due to **higher production budgets and audience demand** for extended content.
  - However, some recent movies still have shorter runtimes, especially in the **animated and documentary genres**.
- 

## □ Final Thoughts

- The **film and television industry has seen massive growth** over the years.
- The **shift towards digital streaming platforms** is reflected in the **increase in content production post-2010**.
- **Genres like Drama and Comedy dominate**, while **shorter content is becoming more frequent**, likely due to social media and online video trends.
- Understanding these trends helps in predicting **future developments in the entertainment industry**.